Pontificia Universidad JAVERIANA Colombia

Flujos con estado en LangGraph

Semana 8 · Chaining, routing y paralelización
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3 ✓

RAG y evaluación
Semanas 4–6 ✓

Model Context Protocol
Semana 7 ✓

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 8 · LangGraph: estado, routing y paralelización

- De `create_agent` a un flujo armado a mano
- Estado, nodos, aristas y reducers
- `MessagesState` y el agente en Studio
- Prompt chaining con una compuerta
- Routing a especialistas
- Paralelización: fan-out y fan-in

Las ideas que traemos de la semana 7

Repaso

Antes de avanzar

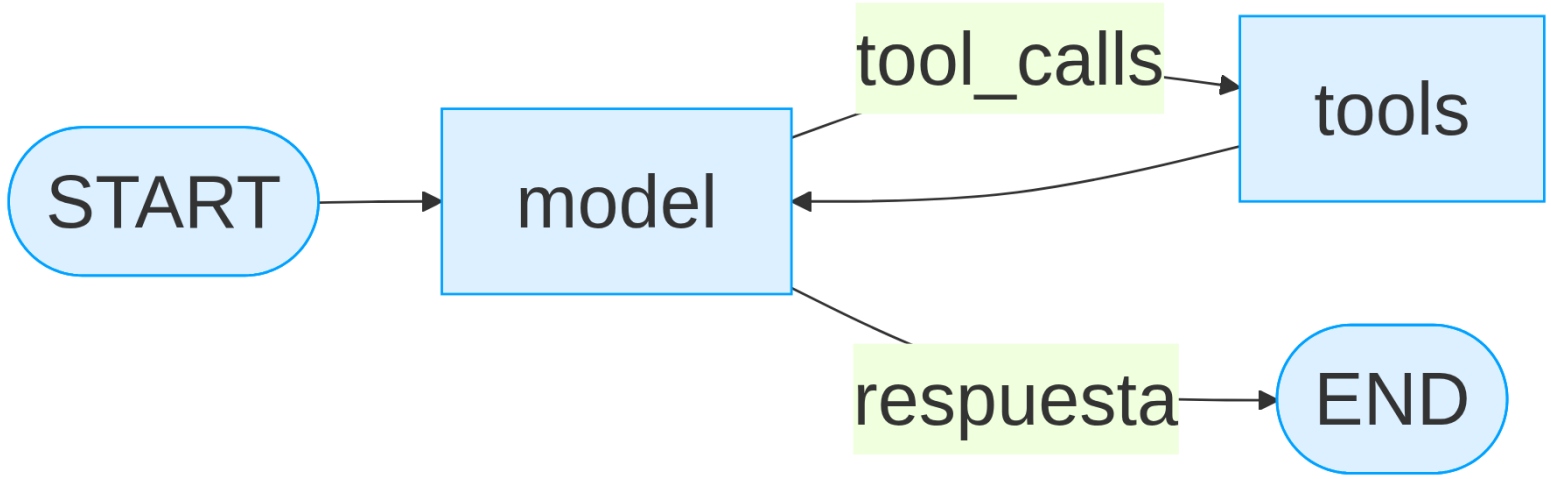
1. MCP separa quién implementa una herramienta de quién la usa.
2. El agente de la semana 3 sigue igual: solo cambian de dónde vienen sus tools.
3. Hasta ahora, `create_agent` decidía el flujo por nosotros.

El ciclo que ya conocemos

create_agent es un grafo de LangGraph

Dos nodos y un ciclo

L14 importa el agente de `src/agents/main.py` y lo abre en Studio con `langgraph dev`.



Workflows y agentes

Qué fija el código y qué decide el modelo

Patrón	El código fija	El modelo decide
Prompt chaining	Los pasos y la compuerta	El contenido de cada paso
Routing	Los especialistas posibles	A cuál va cada pregunta
Parallelization	Los revisores y el agregador	El veredicto de cada revisor
Agente	Las herramientas disponibles	Qué herramienta usar y cuándo parar

Pontificia Universidad JAVERIANA Colombia

Los bloques de LangGraph

L15 y L16

Estado, nodos y aristas

L15 · secciones 1 a 3 · sin modelo

Tres piezas

Un nodo devuelve una actualización parcial del estado.

```
class TicketState(TypedDict, total=False):
    customer: str
    message: str
    category: str
    priority: str

def classify(state: TicketState) -> dict:
    ...
    return {"category": category}  # solo lo que cambia

builder = StateGraph(TicketState)
builder.add_node("classify", classify)
builder.add_node("prioritize", prioritize)
builder.add_edge(START, "classify")
builder.add_edge("classify", "prioritize")
graph = builder.compile()
```

Reducers y mensajes

L15 · sección 4 · L16

Combinar en lugar de reemplazar

1. Sin reducer, el valor nuevo de un campo reemplaza al anterior.
2. `Annotated[list[str], operator.add]` concatena las listas de cada nodo.
3. `MessagesState` usa `add_messages`: agrega mensajes y reemplaza por `id`.
4. `stream` muestra la actualización de cada nodo paso a paso.

Pontificia Universidad JAVERIANA Colombia

Prompt chaining

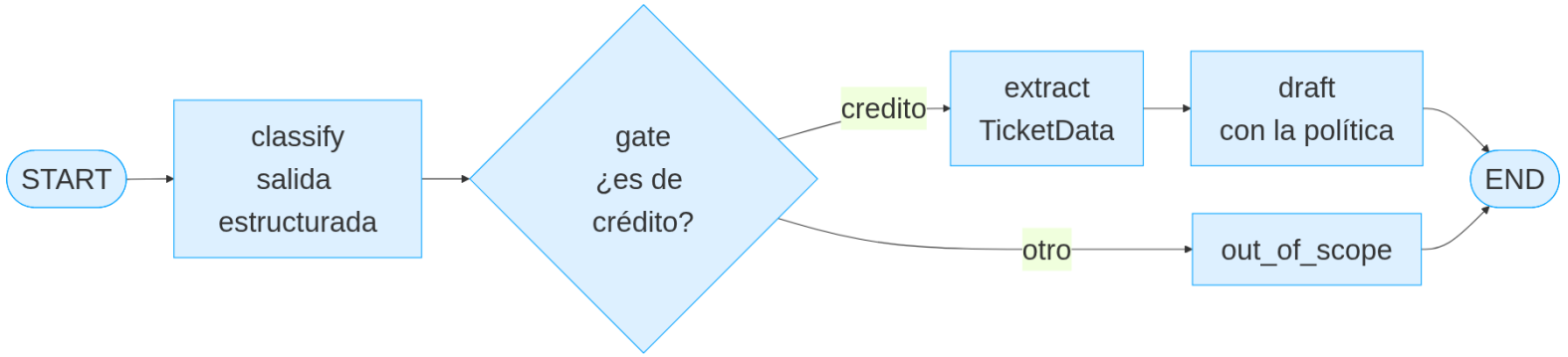
L17 · pasos fijos con una compuerta

Clasificar, extraer, redactar

L17 · 4 llamadas

Una compuerta en código

Si el mensaje no es de crédito, el flujo se detiene sin gastar dos llamadas más.



Cuándo encadenar

Prompt chaining

Pasos conocidos de antemano

1. Cada paso tiene un prompt corto y una salida estructurada.
2. La salida de un paso es la entrada del siguiente.
3. La compuerta es código: comprobable y sin costo.
4. Más pasos significa más latencia: medir cuántas llamadas hace cada camino.

Pontificia Universidad JAVERIANA Colombia

Routing

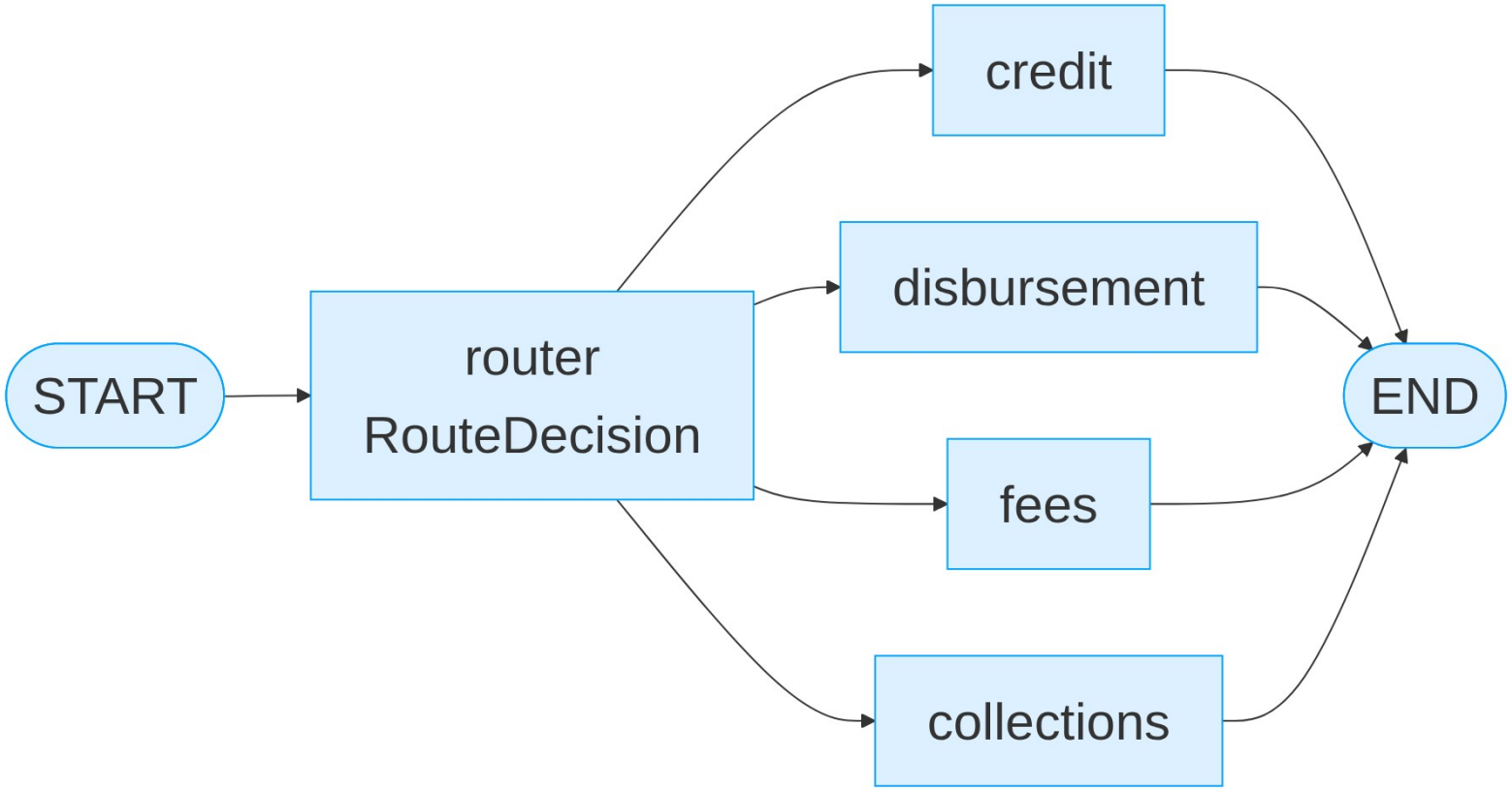
L18 · un clasificador elige el especialista

Un router y cuatro especialistas

L18 · 6 llamadas para 3 preguntas

Prompts cortos y enfocados

Cada especialista recibe solo su política: crédito, desembolsos, costos o cobranza.



De la regla al router

L18 · secciones 1 y 2

Dos formas de decidir

1. Primero, una arista condicional con palabras clave: rápida, gratis y frágil.
2. Después, un router con salida estructurada: `Literal` con las cuatro rutas.
3. La descripción de cada ruta es la instrucción del clasificador.
4. Revisar la ruta elegida para cada pregunta es parte de la evaluación.

Pontificia Universidad JAVERIANA Colombia

Paralelización

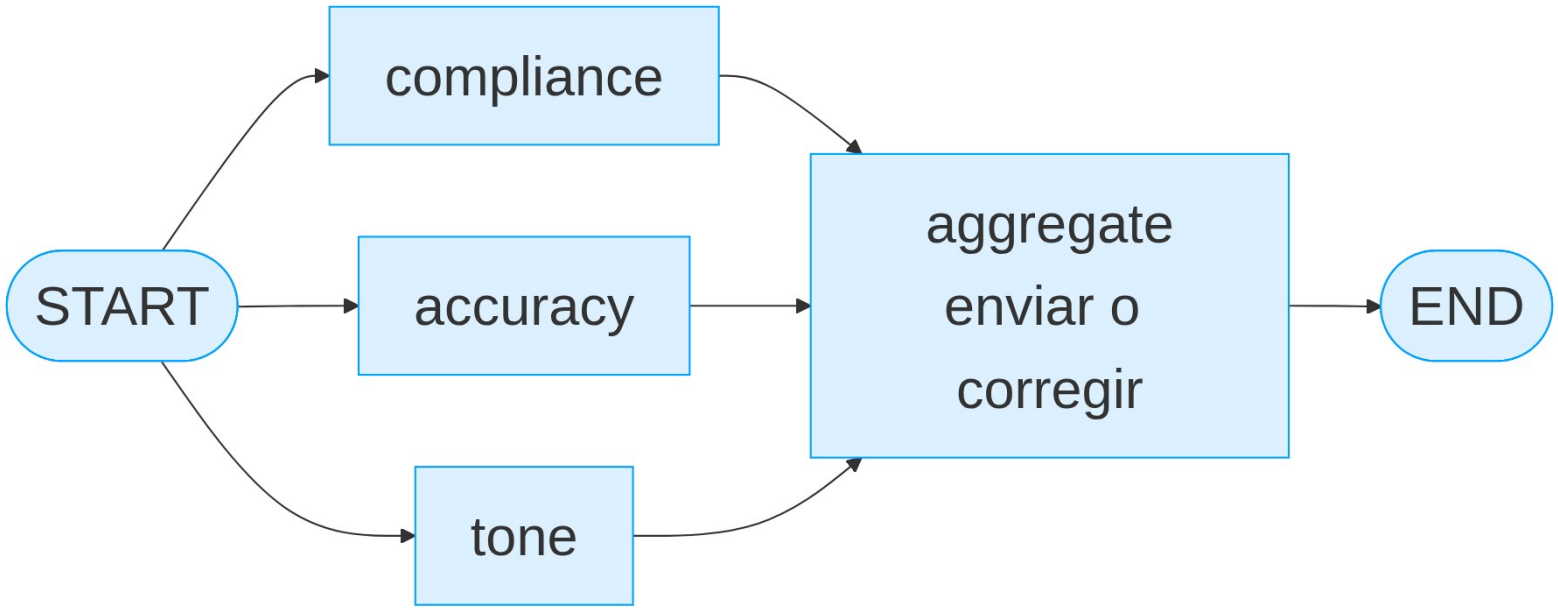
L19 · fan-out y fan-in

Tres revisores en paralelo

L19 · 3 llamadas simultáneas

Antes de enviar una respuesta

compliance, accuracy y tone revisan el mismo borrador; `aggregate` decide en código.



Juntar sin que se pisen

L19 · secciones 2 a 4

Reducer y agregador

1. Los tres revisores escriben en `reviews`, una lista con `operator.add`.
2. El agregador espera a los tres y decide: enviar o corregir.
3. El borrador de prueba promete un periodo de gracia que la política no ofrece.
4. En paralelo, el tiempo total es el del revisor más lento, no la suma.

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L14 a L19

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. L14–L16: el agente en Studio, el estado y los mensajes (2 llamadas).
2. L17: dos mensajes, dos caminos por la compuerta (4 llamadas).
3. L18: tres preguntas, tres especialistas (6 llamadas).
4. L19: revisión en paralelo y comparación de tiempos (3 llamadas).

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Un nodo devuelve solo lo que cambia.
2. Los reducers definen cómo se combinan las actualizaciones.
3. Lo que se puede decidir en código no necesita al modelo.
4. Cada patrón se justifica frente a la alternativa más simple.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 5	Arquitecturas cognitivas con LangGraph
Anthropic, Building effective agents	Chaining, routing y parallelization
Documentación de LangGraph	Workflows and agents · Graph API

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Human-in-the-loop: aprobar un desembolso antes de ejecutarlo.
Notebook: `L20-HumanInTheLoop.ipynb`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso